

Chip Convergence

Verification-first agents for chip design

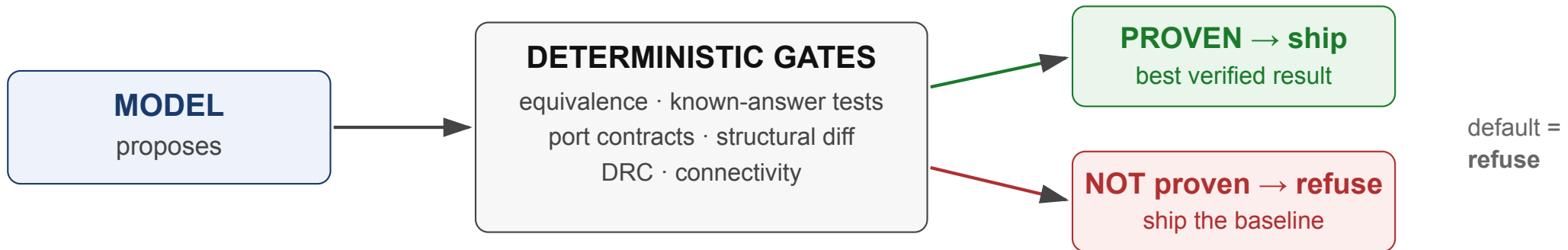
NVIDIA RTL PPA · **NXP** SoC generation · **ASU** DRC repair

ICLAD-DAC 2026 · GenAI Chip Hackathon

Harikrishnan KC (solo) · greatharikrishnan@gmail.com

1 · One thesis, three problems

A language model will hand you hardware that is **smaller, faster — and wrong**.
Plausible-but-wrong RTL is worse than no RTL: it passes review and fails in silicon.
So we built the **verifier first** and put the model behind it.



The model is a **search heuristic over a verified space** — never an authority.
The system's default is to ship nothing.

2 · NVIDIA — improve PPA, stay functionally identical

Given: 7 IPs (async_fifo, sha512, NVDLA, OpenTitan aes/ascon/kmac/prim), their testbenches, a Yosys + OpenSTA ASAP7 flow.

Scored on: area × delay vs baseline (**ADP**, lower is better), gated on correctness.

Five verification layers — every candidate, every round:

#	layer	what it answers
1	lint	structurally sane?
2	compile	does it elaborate?
3	gate	the IP's own testbench — fail-closed: nonzero rc = FAIL
4	LEC	yosys <i>proves</i> equivalence to pristine
5	dualsim	cycle-exact co-simulation, baseline vs candidate

PROVEN binds all of: `rc==0` + success line + `total>0` + `proven==total` + `unproven==0` .

Anything else is INCONCLUSIVE — never a pass.

Canonical = LEC-PROVEN. An unproven candidate is never shipped, however good its ADP.

3 · NVIDIA — results, and the ones we refused

IP	ADP	assurance
prim	0.5824	LEC PROVEN + dualsim · power -70%
sha512	0.7266	full 5-layer · timing -97 → +335 ps (MET)
ascon	0.97918	LEC PROVEN + dualsim
aes	1.0001	differential-only — honestly <i>"explored-no-adp-gain"</i>
kmac	1.0	bounded negative — headroom is inside the masked core
async_fifo	1.0 (baseline)	deliberately reverted — next slide

Two refusals worth more than the wins:

- **ascon**: a candidate *beat* the banked ADP — `lec_diagnostic` found a real counterexample → **refused**. The gate is load-bearing, not decorative.
- **sha512, live**: a candidate at **ADP 0.6546** — far better than anything we ship — was **refused for not being LEC-PROVEN**. The system gave up a 35 % headline number rather than ship something it could not prove.

4 · NVIDIA — the 4 % win that was the bug

Our agent found a **4 % ADP win** on an asynchronous FIFO.

It passed lint. Compile. The testbench. Formal equivalence **PROVED** it. Cycle-exact differential simulation passed. **Five for five** — our best-verified candidate.

It was broken.

It removed the registers on the **Gray-code pointer** and computed `gray(bin)` combinatorially. The logic function is identical — *which is why formal proved it*. But at a **clock-domain crossing** a combinational encoder **glitches** mid-transition, and the receiving domain samples asynchronously. Intermittent pointer corruption.

We reverted to the registered baseline, then rebuilt the *separable* part of the optimization on the safe base and measured it: **ADP 0.9984 — noise**.

The entire 4 % "win" was the hazard.

5 · NVIDIA — NVDLA: whole-design formal at scale

~950,000 cells. Whole-design equivalence **PROVEN: 381,209 / 381,209** `$equiv` cells through the production recipe — uncommon at this scale.

Release packet: 6/6 — every gate control demonstrated live, including the negatives:

control	result
pristine gate must PASS	✓ 1 passed / 0 failed
mutant gate must FAIL	✓ 0 passed / 1 failed
LEC positive PROVEN · LEC negative not-proven	✓ ✓
candidate-survival tripwire · determinism ×2	✓ ✓

We found our own tooling lying. The packet sat at 5/6 for a day. The failing check was a bug in *our log parser* — it counted the word "failed" inside the runner's help banner (`exceeded => failed`) as a test failure. Fixed, re-run, 6/6 — **and the negative control still fires**, proving the fix did not simply blind the gate.

Not claimed: an NVDLA PPA result. The campaign plumbing is unfinished; on a hidden NVDLA-like case the agent attempts, cannot prove, and **ships baseline**.

6 · NXP — generate a secure SoC from a block diagram

Given: an architecture document and a testbench port skeleton. **Produce synthesizable RTL.**

Model reads the diagram → emits **YAML specs per IP** → `rtl_gen_lib` generates Verilog → model stitches the top → **correctness firewall** → gate.

The firewall — deterministic, non-bypassable, ≤ 3 error-fed re-prompts:

- **YAML validator** — typed errors *before* any RTL is generated
- **Port contract** — token-level diff of the top's ports vs the skeleton's DUT instantiation
- **Reset lint** — raw POR may feed *only* the synchronizer; one common synchronized net
- **Structural diff** — instance census, IRQ reachability, bus connectivity
- **Port-direction gate** — an instance's output may never be driven by a constant

Result (easy tier): 2 model calls, ~42 s — GATE 30/30, KAT 79/79 against a golden oracle *and* **79/79** against the model's own oracle, STG differential cycle-identical over **3,662** cycles.

Two oracles on purpose: self-consistency proves the model agrees with itself; the golden oracle proves it is right.

7 · NXP — the night the hidden problems landed

The organizers released **medium** (2×3 TileLink NoC AES SoC) and **hard** (multi-domain crypto SoC: 4×3 NoC, 4 AES, 2 DMA) — architectures the agent had never seen.

Running them exposed **nine defects in our own agent**. The worst three:

defect	effect
top-module name hardcoded to the easy problem	every other tier failed elaboration
prompt budget sized for 8 IPs	14 of 22 IP interfaces never shown → the model <i>guessed</i> ports
<code>top_ports</code> parsed <code>#{parameter...}</code> as the port list	any parameterised module reported zero ports — silently disabling direction checks

After the fixes — verified with `iverilog` outside the agent:

tier	top derived	contract	elaborates
easy	<code>secure_periph_soc</code>	clean, attempt 1	✓ 30/30 + KAT 79/79 ×2
medium	<code>noc_aes_soc</code>	clean, attempt 3	✓
hard	<code>crypto_soc</code>	clean, attempt 2	✓

8 · NXP — instructions are probabilistic. Gates are not.

A three-step experiment, run last night:

1. The model already had the information.

The port directions were *in the prompt*: `output wire [1:0] p0_d_param.`

It tied constants to those outputs anyway — illegal Verilog, elaboration dies.

2. We added an explicit prohibition.

"An OUTPUT port may NEVER be connected to a constant."

It complied on **medium** (2 runs of 2). It **violated it on hard**.

3. We added a deterministic gate.

`instance_port_directions` compares every connection against the real port direction.

Caught **4/4** violations. The typed error feeds the existing repair loop — and medium and hard then reached contract-clean at attempts 3 and 2.

The model repaired its own output once the gate told it exactly what was wrong.

Plus a mechanical last resort — `.m_rready(1'b1)` → `.m_rready()` — so a stubborn violation degrades to an elaborating design instead of a zero.

9 · NXP — what we do and do not claim

Claimed, and reproducible:

- easy tier: 2 calls, 30/30 gate, KAT 79/79 on both oracles
- medium and hard: contract-conformant RTL that **elaborates against the organizer's skeleton**, on architectures the agent had never seen
- stdlib-only agent; a `_yaml_compat` shim that engages only when PyYAML is absent, parses all 8 reference specs byte-identically, and returns an **inert string** for `!!python/object`

Not claimed — and this matters:

- **No golden testbench ships for *any* tier**, easy included. So medium and hard cannot be *scored*. "Elaborates and conforms" ≠ "functionally correct".
- The official runner still hardcodes `PROBLEMS = ["easy"]`; we drove the new tiers through the same `Paths.from_info` contract the runner uses.
- Our easy result is **"perfect against our verification stack"** — the organizer's hidden testbench is not in the public checkout.

10 · ASU — repair DRC violations in a layout block

Given: a KLayout `pya` layout script, its DRC report, connectivity and design rules.

Scored on: final-violation-rate (**FVR**, lower is better), gated on the repair being valid and connectivity-preserving.

The result that shows judgment: we took the model out.

The winning repair is **deterministic** — replace each flagged multi-cut via array with one continuous via **bar**, derived directly from the rule. No model call at runtime.

Why that was forced, and why it was right: the official agent image is

`python:3.10-slim` with **no KLayout binary**, run `--read-only`. Our development agent measured every candidate with the real evaluator to keep-best — impossible there.

So the submission agent applies the proven transform and emits an artifact **byte-identical** to the independently re-scored development output.

We used the model to *find* the transform. We did not need it to *apply* the transform.

11 · ASU — the hidden blocks, scored

The organizers released the previously-hidden blocks (**Block4**, **Block5**) the night before this talk. We ran the **unmodified** submission agent and scored all seven blocks in one consistent run with the pinned `asu-klayout:0.30.1` evaluator.

block	kind	valid	conn	FVR	repair rate
Block1	public	✓	✓	0.729508	0.590164
Block2	public	✓	✓	0.764706	0.588235
Block3	public	✓	✓	0.764045	0.505618
Block6	public	✓	✓	0.676113	0.655870
Block7	public	✓	✓	0.682353	0.584314
Block4	HIDDEN	✓	✓	0.693878	0.619048
Block5	HIDDEN	✓	✓	0.794118	0.514706

7/7 eligible · public mean **0.7233** · hidden mean **0.7440** — ~3 % apart · no-op ≈**1.25–1.32**

Credible, not luck: the five public scores reproduced to **six decimal places** against values recorded weeks earlier — same harness that produced the hidden numbers.

12 · ASU — why it generalized

No tuning. No retraining. No code change. The hidden blocks were released and the agent scored them in the same band as the public ones.

Because the repair is derived from the rule, not fitted to the data.

A model that had been *optimized against the five public blocks* would have no reason to transfer. A transform decoded from the DRC rule itself has every reason to.

Two honesty points we put on the slide before anyone asks:

- **No-op is not FVR 1.0.** FVR is the evaluator's *fresh* DRC total ÷ the *supplied reference* total, and those are not count-identical: public no-op FVR is **≈1.25–1.32**.
"No-op" describes behaviour, not a score.
- **Net win, not "can never be worse".** Some `V4.AUX.1` / `V5.AUX.1` violations *appear* while the total falls. It was a net scoring win on all five public blocks — that is the claim, and it is the only claim.

13 · Three domains, one finding

	the model was told	what happened	what worked
NVIDIA	—	a candidate passed all five layers incl. LEC-PROVEN and was still unsafe (CDC glitch)	a structural invariant , not more checking
NXP	port directions, in the prompt	ignored them; obeyed an explicit rule on one problem, broke the other	a deterministic gate — 4/4, every time
ASU	—	hidden blocks scored in-band, untouched	the transform is derived from the rule , not fitted

Deterministic gates beat model instructions.

That is what makes **unseen inputs** survivable — and unseen inputs are the whole game.

14 · What formal and simulation structurally cannot see

LEC and zero-delay simulation reason in a model where **glitches and physical side-channels do not exist**. Two classes of edit therefore pass every functional check and are still broken:

1 · Side-channel masking removal — aes S-box, kmac DOM-masked Keccak.

Un-masking preserves the logic function *exactly*, so **LEC returns PROVEN** while the countermeasure is destroyed. Formally equivalent, cryptographically broken.

2 · CDC glitch hazards — `async_fifo` (slide 4).

Neither formal nor zero-delay simulation can represent a glitch.

You cannot fix this by adding more checking.

The bug lives **outside the model the checkers use**. The answer is **structural policy**: per-IP forbidden edit zones (FENCES — implemented, mandatory for NVDLA), and a registered-crossing invariant for CDC.

Honest status: the FENCES are enforced in tooling; the CDC invariant was implemented this week after we caught the agent re-selecting the candidate we had manually reverted — by running our own submission the way a judge would.

15 · Every claim, one command

track	headline	verify it
NVIDIA	prim 0.5824 , sha512 0.7266 , both LEC-PROVEN	<code>submission/<ip>/manifest.json</code> — per-layer assurance is recorded per artifact
	NVDLA formal 381,209/381,209 ; packet 6/6	<code>python3.12 -m ppa.release_control --ip nvdla</code>
NXP	easy 2 calls · 30/30 · KAT 79/79 ×2	<code>python3 nxp_agent.py --model stub</code>
	medium + hard elaborate , unseen	<code>iverilog -g2005 <rtl>/*.v <tb_top_skeleton.v></code>
ASU	7/7 eligible, hidden mean FVR 0.7440	<code>evaluator/evaluate_repair.py --case Block4</code>

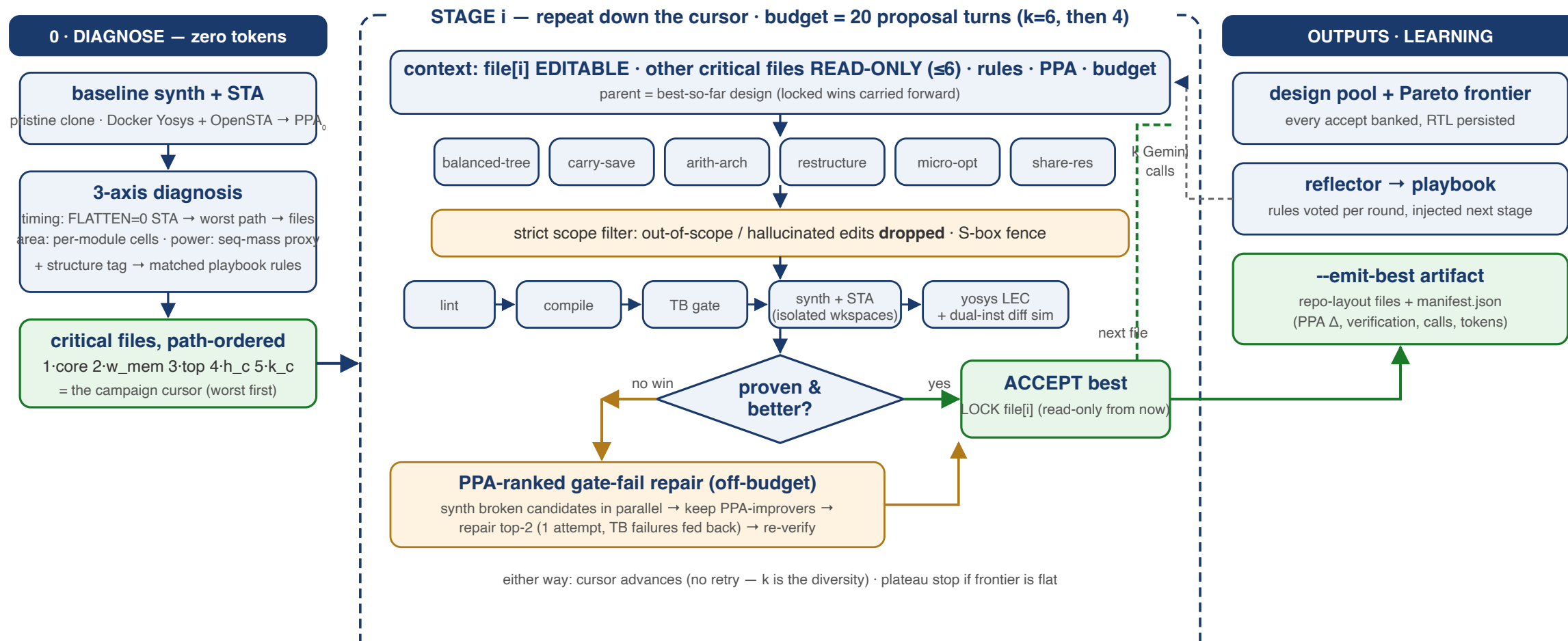
Reproducibility is the point. NXP and ASU reproduce byte-identically. The NVIDIA *artifacts* re-synthesize exactly; re-running the *search* is stochastic — and we say so.

The architecture, in one line: the model proposes, deterministic gates dispose, and the default is to ship the baseline.

Backup

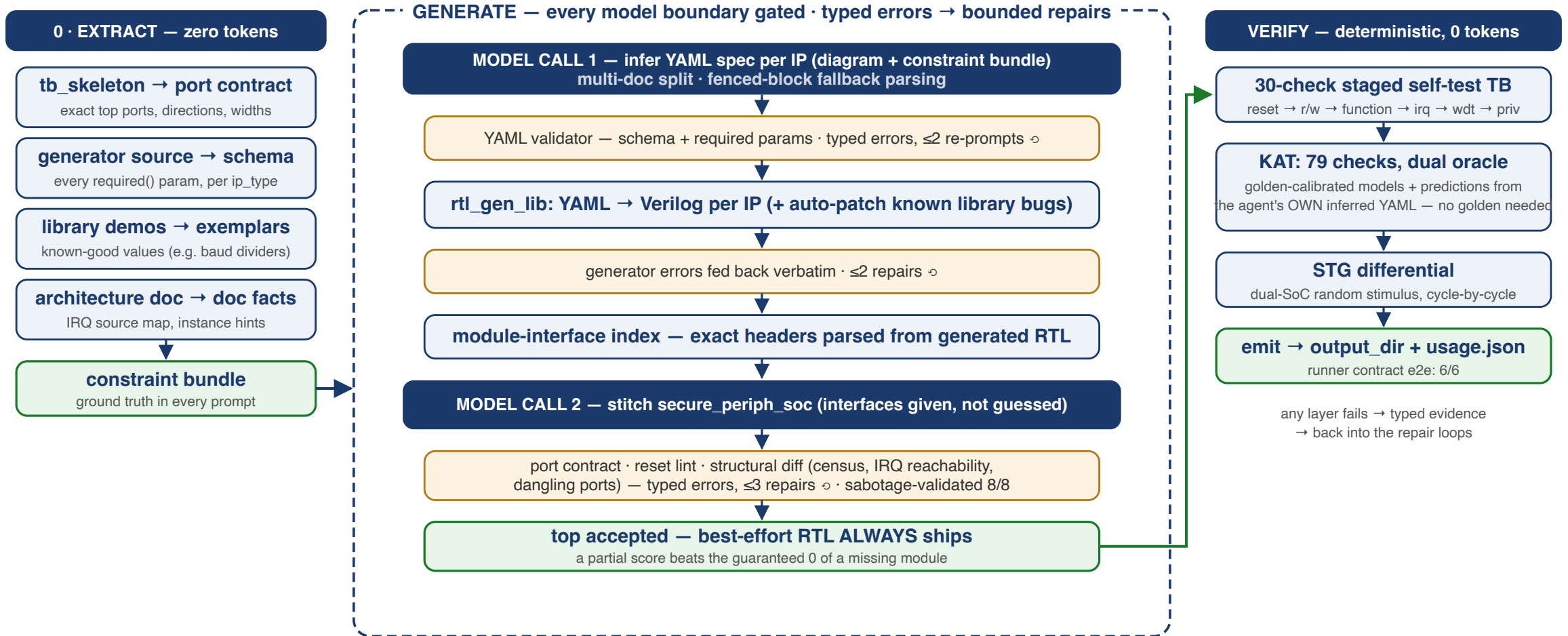
Flow diagrams · number sheet · what we do not claim

B1 · NVIDIA — the full flow



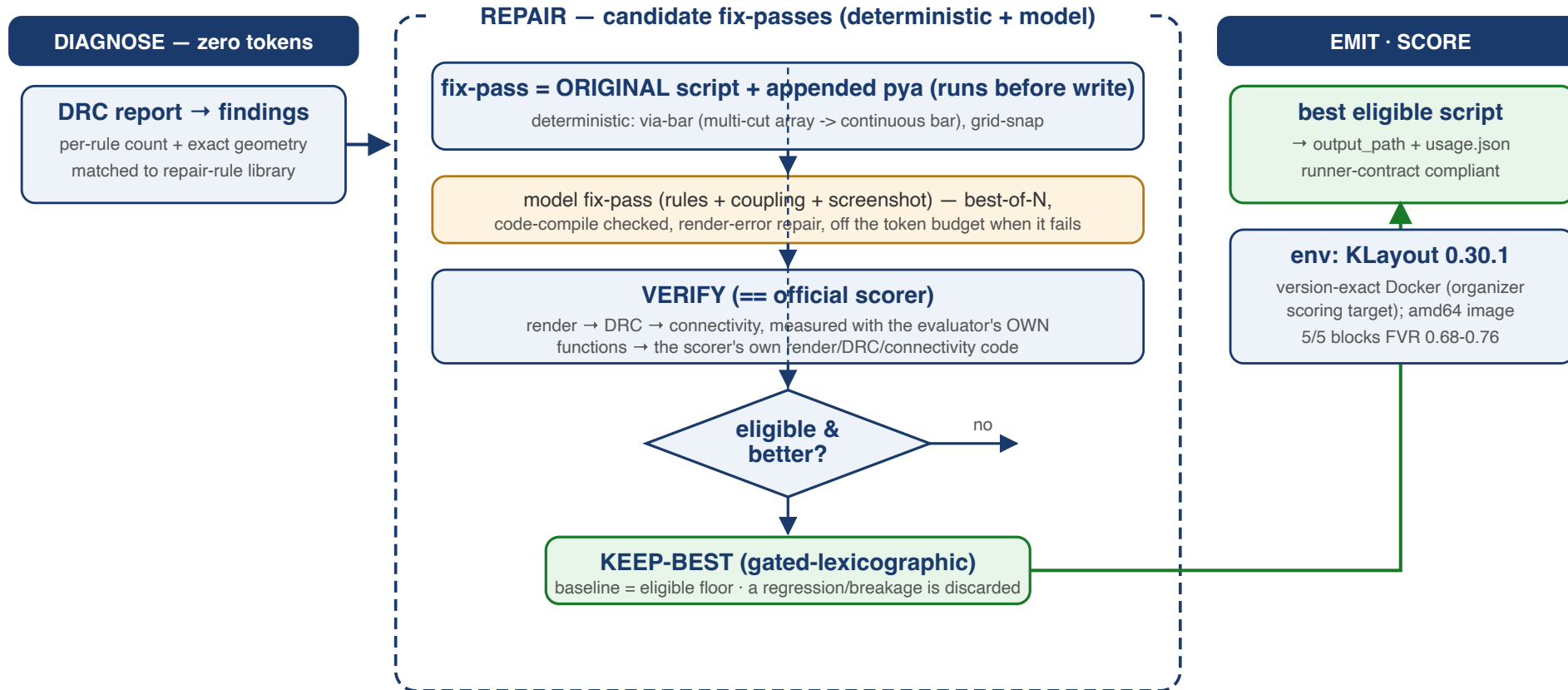
Banked: sha512 **ADP 0.7266** full 5-layer · prim **ADP 0.5824** LEC-PROVEN + dualsim (power -70%) · async_fifo ships **baseline** by design.

B2 · NXP — the full flow



Live: **2 model calls, 42 s** → 30/30 · KAT 79/79 (both oracles) · STG 3662 cycles, **0 differing** vs hand-built reference.

B3 · ASU — the full flow



Same verification-first spine as NVIDIA/NXP: diagnose (0 tokens) → propose → verify == scorer → keep-best (credibility-gated) → emit. **FVR 0.68–0.76 on all 5 blocks.**

B4 · Verified number sheet

NVIDIA (ADP, baseline = 1.0, lower better)

prim **0.5824** LEC-PROVEN+dualsim, power -70% · sha512 **0.7266** full 5-layer, WNS -97 → +335 ps

ascon **0.97918** PROVEN · aes **1.0001** differential-only · kmac **1.0** bounded negative

async_fifo **1.0** deliberate revert · NVDLA formal **381,209/381,209**, release packet **6/6**

Refused: ascon INEQUIVALENT counterexample; sha512 **0.6546** unproven → refused

NXP easy: **2** calls / ~42 s · GATE **30/30** · KAT **79/79** golden + **79/79** model ·

STG differential **3,662** cycles · medium (`noc_aes_soc`) and hard (`crypto_soc`) elaborate,

contract-clean at attempts 3 and 2

ASU 7/7 eligible · public FVR 0.676–0.765 (mean **0.7233**) ·

hidden **Block4 0.693878**, **Block5 0.794118** (mean **0.7440**) · no-op reference ≈**1.25–1.32**

Reproduction, 2026-07-24/25: fresh GitHub clone → NXP stub 30/30 + KAT 79/79×2; ASU output

byte-identical (`fdae65dd...`) under `python:3.10-slim --read-only` ; NVIDIA prim re-run

improved to **0.5762** LEC-PROVEN.

B5 · What we deliberately do NOT claim

- **Not claimed:** an NVDLA optimization result. Whole-design formal proves; the campaign plumbing is unfinished and the contract stays `PENDING` — every real model campaign is refused.
- **Not claimed:** an official NXP hidden-testbench score. **No golden testbench ships for any tier.** medium/hard "elaborate and conform" $\neq$ "functionally correct".
- **Not claimed:** ASU "can never be worse" — it was a net win on all five public blocks; some individual violation classes appear while the total falls.
- **Not claimed:** full 5-layer assurance on prim or aes — their manifests record `gate: SKIP-preexisting` and label the result **equivalence+differential**.
- **Not claimed:** "no model can improve kmac" — bounded probes found nothing; that is a bounded negative, not a universal.
- **Not claimed:** run-to-run identity of the NVIDIA search. A shallower run legitimately produced sha512 0.7814 where a deeper one produced 0.7266. The *artifact* reproduces; the *search* is stochastic.

Every one of these was, at some point, stated too strongly in our own documents — and corrected. We audit our own claims.